

## Titre

Développement et partage d'un package Python : transformer un code individuel en un package réutilisable par la communauté. Illustration sur un solveur symbolique pour la dynamique non-linéaire.

## Résumé

Le développement de codes numériques occupe une place centrale dans la recherche en mécanique des structures. Les chercheurs et doctorants investissent une grande partie de leur temps dans la construction d'outils efficaces, souvent aboutis lorsque la thèse s'achève ou le projet de recherche se conclut. L'enchaînement des projets laisse peu de temps à une étape de mise à disposition de ces codes (documentation, versionnage, archivage), ce qui les rends difficilement réutilisables. C'est une perte considérable, tant pour la continuité des projets au sein des laboratoires que pour la communauté scientifique au sens large, qui peine à reproduire des résultats ou à comparer des méthodes sans les réimplémenter entièrement.

Ce workshop aborde cette problématique sous l'angle du partage de code sous forme de package Python. Au-delà de l'aspect technique, il s'inscrit dans une démarche de science ouverte : un code bien structuré, documenté et publié devient une production scientifique à part entière (citable, archivée, réutilisable) qui renforce la diffusion des méthodes développées et leur adoption par d'autres équipes, académiques ou industrielles.

La session s'organise en deux temps. Une première partie présente les grands axes qui font d'un code un package partageable : architecture du code, documentation, gestion de version avec Git et dépôt GitHub, génération de documentation en ligne, archivage pérenne et citation, publication sur l'index de packages PyPI. Ces concepts sont illustrés sur *oscilate*, un package Python implémentant la méthode des échelles multiples pour la résolution symbolique d'équations de dynamique non-linéaire. La seconde partie est consacrée à la mise en pratique : chaque participant construit et partage son propre package, à partir de son code existant ou d'un exemple fourni.

## Instructions pré-workshop

- Venir avec son PC
- Installer Python (pas via Anaconda, directement ici <https://www.python.org/downloads/>)
- Installer VSCode : <https://code.visualstudio.com/download>
- Installer les extensions suivantes dans VSCode
  - autoDocstring - Python Docstring Generator
  - GitHub Actions
  - Graphviz (dot) language support for Visual Studio Code
  - Jupyter
  - Pylance
  - Python
  - Python Debugger
  - TOML Language Support
- Tester l'installation de python
  - Créer un script minimaliste nommé `script_test.py`, comme celui de cet exemple : <https://code.visualstudio.com/docs/python/jupyter-support-py>
  - Depuis le terminal VSCode, lancer `python script_test.py`.

- Si le test n'est pas concluant, python n'est pas accessible depuis la variable système PATH, il faut l'ajouter. Pour Windows (peu de problèmes avec Mac et Linux)
  - Ajouter les chemins suivants à PATH  
C:\Users\<username>\AppData\Local\Programs\Python\Python3xx\  
C:\Users\<username>\AppData\Local\Programs\Python\Python3xx\Scripts\  
Voir procédure ici : <https://lecrabeinfo.net/tutoriels/modifier-le-path-de-windows-ajouter-un-dossier-au-path/>
  - Relancer le terminal.
- Lancer les cellules de script\_test.py dans la fenêtre interactive python, comme dans cet exemple : <https://code.visualstudio.com/docs/python/jupyter-support-py>
- Télécharger le .zip contenant les fichiers nécessaires au workshop (bientôt disponible)